SentinelIQ 10 — Threat Model

Oct 13, 2026 · @maxi

This threat model covers attacks on SentinelIQ itself, because a security tool that can be tampered with is worse than none.

Scope and assets

In scope: the API, worker, database, frontend, CI/CD pipeline, secrets, and the LLM integration. **Out of scope:** the cloud provider's own infrastructure, analysts' laptops, and physical security.

The data is synthetic or public, but the threat model treats it as if it were real, because that is the habit a production system needs.

Asset	Why it matters	Property to protect
User credentials and tokens	They open access to alerts and decisions	Confidentiality
Event data	May contain IP addresses and usernames	Confidentiality, integrity
Audit log	Evidence of what analysts decided and why	Integrity
Model files and metrics	A swapped model silently changes every score	Integrity
LLM API key and other secrets	Leakage means cost and data exposure	Confidentiality
Explanations	Misleading text can lead to a wrong decision	Integrity

Attackers and trust boundaries

Who might attack

- An unauthenticated outsider probing the API or flooding ingestion.

- A logged-in user trying to read or change what their role does not allow.
- An attacker who controls text inside events and hopes it reaches the LLM as an instruction.
- A compromised dependency or container image in the supply chain.
- Anyone with repository access who could leak a secret.

Trust boundaries

- Internet to API: every request is untrusted until authenticated and validated.
- API to database: only parameterized queries; the audit table is insert-only.
- Worker to LLM provider: event content leaves the system here, so only the minimum is sent.
- CI to production: only a passing build on the main branch may deploy.

Threat register

Threats are grouped by the STRIDE categories. Status shows the phase in which each control is built.

ID	Threat	Category	Component	Mitigation	Status
T1	Stolen or forged JWT	Spoofing	API	Short-lived tokens, signature checks, signing secret kept in a secret store	Planned, Phase 3
T2	User reaches a route above their role	Elevation of privilege	API	Role check on every route, deny by default, a test per role	Planned, Phase 3
T3	SQL injection through event fields	Tampering	API, database	Parameterized queries only, schema validation	Planned, Phase 3
T4	Prompt injection through log text	Tampering	Worker, LLM	Controls in Document 03, plus the adversarial test set	Planned, Phase 5
T5	Flood of fake or duplicate events hides real attacks	Denial of service	Ingestion API	Rate limits, payload size limits, deduplication, capped job retries	Planned, Phase 3; tested Phase 7
T6	Analyst decision altered or deleted	Tampering, repudiation	audit_log	Insert-only database permissions, no update routes, request IDs	Planned, Phases 3 and 6

ID	Threat	Category	Component	Mitigation	Status
T7	Secret committed to the repository or written to logs	Information disclosure	Repository, CI, logs	Environment variables, secret scanning in CI, log scrubbing	Planned, Phase 2
T8	Vulnerable dependency or container image	Tampering	Build	Dependency audit, image scan, pinned versions	Planned, Phase 2
T9	Swapped or poisoned model file	Tampering	Model store	Hash stored in `model_versions` and verified on load, restricted write access	Planned, Phase 4
T10	Analyst trusts a misleading explanation	Integrity	Explanations	Sources and confidence shown, failed explanations flagged, human decision	Planned, Phases 5 and 6
T11	Event content sent to the LLM provider	Information disclosure	Worker	Send the minimum fields, review provider terms, no real data in this project	Planned, Phase 5
T12	Analyst decisions used as labels poison retraining	Tampering	Retraining (SH3)	Not built in v1; if added, labels are reviewed before retraining	Deferred

Residual risks

These risks remain after the planned controls, and the README should state them.

- A prompt-injection attempt may still produce a wrong explanation; visible sources and the human decision are the last line of defence.
- The LLM provider sees event content. This is acceptable only because the data is public or synthetic; real data would need a provider agreement and further data minimization.
- One database is a single point of failure in v1, and backup and recovery are out of scope.
- The benchmark data may not represent live traffic, so model quality in real use is unknown.
- No independent penetration test is planned; the project relies on its own tests and automated scanning.

Security testing plan

Each control in the register gets a check that fails the build when the control breaks.

Check	Covers	Where it runs
Role-by-route test matrix: every role against every route	T1, T2	CI, integration tests
Malformed and hostile input tests against ingestion	T3, T5	CI, integration tests
Adversarial prompt set with a reported pass rate	T4, T10	CI with recorded responses; scheduled run against the live provider
Audit-log tamper test: update and delete must fail at the database	T6	CI, integration tests
Secret scan of the repository history	T7	CI
Dependency audit and container image scan	T8	CI
Model hash verification on load	T9	CI, unit tests
Load test with a flood of duplicate and oversized events	T5	Phase 7

A short written security review closes Phase 7: each threat in the register is marked mitigated, accepted or deferred, with the evidence.